

Reviewer Pack — Minimal Order of Formal Grammar Entailment and Circuit Monot...

Entry 18e18ca2ff4a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 20:15:24 UTC

Minimal Order of Formal Grammar Entailment and Circuit Monotone Width Inequality

Entry ID: 18e18ca2ff4a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 20:15:24 UTC

1. Conjecture

Field A (mathematical branch): Formal Logic (Entailment Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For any given formal grammar G with a set of production rules that generate clauses of a SAT instance, the minimal order of entailment $\omega(G)$ is linearly correlated with its associated circuit's monotone width $w_c(G)$, such that $\omega(G) = \Theta(w_c(G))$.

Rationale (proposer's reasoning):

Formal logic provides a framework to reason about logical implications, which can be used to predict the complexity of computation. The minimal order of entailment quantifies the depth of entailment in a grammar, while circuit monotone width measures the nonlinearity of a function represented by a circuit. A correlation between these two measures could reveal connections between logical reasoning and computational complexity.

Taxonomy category: ENTAILMENT_LOGIC_CIRCUIT_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3a28311543fac11c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a formal grammar G , if the correlation coefficient between minimal order of entailment $\omega(G)$ and monotone width $w_c(G)$ is ≥ 0.8 AND $|\omega(G) - \Theta(w_c(G))| \leq 3$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_satisfiable_formula(n):
        literals = set(range(1, n + 1))
        clauses = []
        for _ in range(n):
            clause = random.sample(literals, random.randint(1, n // 2))
            clauses.append(clause)
        return clauses

    def calculate_minimal_order_of_ entailment(clauses):
        # Simplified heuristic to estimate minimal order of entailment
        return len(max(clauses, key=len))

    def calculate_monotone_width(clauses):
        # Simplified heuristic to estimate monotone width
        return max(len(set(c)) for c in clauses)

    n = 20
    formula = generate_satisfiable_formula(n)
    omega_G = calculate_minimal_order_of_ entailment(formula)
    w_c_G = calculate_monotone_width(formula)

    metric_name = "correlation_coefficient"
    metric_value = (omega_G - w_c_G) / max(1, abs(w_c_G))
    instances_tested = 1
    n_max = n
    conjecture_holds = False
    counterexample = ""

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
```

```

        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f214fde1.py", line 67, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f214fde1.py", line 38, in run_trial

formula = generate_satisfiable_formula(n)

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f214fde1.py", line 25, in generate_sati

clause = random.sample(literals, random.randint(1, n // 2))

^^

File "/usr/lib/python3.12/random.py", line 413, in sample

raise TypeError("Population must be a sequence. ")

TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Review and debug the test code to ensure it can run to completion and produce the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14530
2	propose	ollama_remote	glm4:latest	0	0	12481
3	propose	ollama_remote	glm4:latest	0	0	13647
4	propose	ollama_remote	glm4:latest	0	0	13449
5	preregistration	ollama_remote	glm4:latest	0	0	9444
6	novelty	ollama_remote	glm4:latest	0	0	8376
7	novelty	ollama_remote	glm4:latest	0	0	8501
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55848
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9042
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7138
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8624
12	judge	ollama_remote	glm4:latest	0	0	11710

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172789 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/18e18ca2ff4a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/18e18ca2ff4a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/18e18ca2ff4a.tar.gz` (if generated)